Automated Web Security & HTTP Header Scanner

Cover & Metadata

Field	Detail
Project Title	Automated Web Security & HTTP Header Scanner
Topic Reference	Topic 2 — Automated Web Scanner: Programmed a Python script to audit sites for secure headers and CORS flaws.
Domain	Web Application Security & Defense
Author	Sania Sheikh
Submission Date	October 25, 2026
Tools & Technologies	Python 3.10+, Streamlit ≥ 1.28, Requests ≥ 2.31, Pandas ≥ 2.0, IBM Bob
Primary Source File	`WebSecurityScanner.py`

Table of Contents

1. Abstract & Problem Statement
2. System Architecture & Methodology
3. Role of IBM Bob in Development
4. Testing & Experimental Results
5. Remediation Directives & Hardening Guide
6. Conclusion & Future Scope
7. References

---

1. Abstract & Problem Statement

1.1 Abstract

This report documents the design, implementation, and empirical validation of an automated, passive web security auditing tool developed in Python using the Streamlit framework. The tool inspects any publicly reachable HTTP or HTTPS endpoint for the presence and correctness of seven defensive HTTP response headers, evaluates Cross-Origin Resource Sharing (CORS) policy configuration,

computes a letter-grade security posture score (A through F) via a weighted algorithm, surfaces per-header hardening directives for both Nginx and Apache web servers, and exports a structured audit trail in JSON or CSV format. The project was developed in active partnership with IBM Bob, an AI developer copilot, which contributed to UI scaffolding, HTTP connection logic, bug resolution, and remediation content generation throughout the development lifecycle.

1.2 Problem Statement

The HTTP response header layer is one of the most consistently under-configured security surfaces in modern web deployments. Unlike application-layer vulnerabilities — such as SQL injection or broken authentication — that require attacker skill and application-specific knowledge to exploit, missing security headers require no exploitation at all. Their absence simply removes a browser-enforced control that would otherwise protect end-users. The specific risks posed by each missing header are as follows.

Strict-Transport-Security (HSTS) — SSL Stripping

Without an HSTS header, a browser accepts HTTP connections even when HTTPS is available. An adversary performing a man-in-the-middle (MITM) attack on a shared or hostile network can intercept an initial plaintext HTTP request and silently strip the TLS upgrade — a technique known as SSL stripping. The victim communicates over unencrypted HTTP while the attacker proxies content over HTTPS to the server, remaining entirely invisible. HSTS prevents this by instructing the browser to enforce HTTPS for the duration of the `max-age` period, even before the server is contacted.

Content-Security-Policy (CSP) — Cross-Site Scripting

A CSP header instructs the browser to enforce a source-allowlist for scripts, styles, fonts, images, and other subresources. Without it, any injected `<script>` tag — whether via a stored XSS vulnerability, a compromised third-party CDN, or a malicious browser extension — executes without restriction in the context of the page. CSP is the primary browser-side mitigation against Cross-Site Scripting, ranked by OWASP in the Top 10 most critical web application security risks. A missing CSP is therefore classified as a critical absence in this tool.

X-Frame-Options — Clickjacking

When a page omits this header, an attacker can embed it inside a transparent `<iframe>` overlay on a malicious site. The victim interacts with what appears to be a legitimate UI but is actually manipulating the invisible framed page, allowing an attacker to hijack clicks, form submissions, and authentication actions. This technique is known as clickjacking or UI redressing.

X-Content-Type-Options — MIME Sniffing Attacks

Without the `nosniff` directive, legacy browsers and certain Chromium builds perform MIME-type sniffing — inferring a resource's content type from its byte sequence rather than its declared `Content-Type` header. An attacker who can upload a file that is parsed as valid JavaScript (e.g., a specially crafted JPEG that also satisfies a JavaScript grammar) can cause it to execute as a script in a victim's browser.

Referrer-Policy — Sensitive Data Leakage via Referer

Without a restrictive referrer policy, full URLs — including query strings that may contain session tokens, user identifiers, or confidential search terms — are transmitted in the `Referer` request header to every third-party resource loaded by the page. This leaks sensitive navigation context to analytics providers, advertising networks, CDNs, and any network intermediary.

Permissions-Policy — Unauthorised Browser API Access

This header explicitly restricts which browser APIs (camera, microphone, geolocation, payment, USB) a page and its embedded `<iframe>` elements may access. Its absence means embedded third-party scripts or frames inherit full API access, enabling covert sensor activation, payment-form skimming, or geolocation tracking without user awareness.

Loose CORS Policy — Cross-Origin Credential Theft

A wildcard `Access-Control-Allow-Origin: ` policy instructs every browser to allow any origin to read the response body of cross-origin requests to the server. When combined with endpoints that reflect session cookies or other user-specific state, this enables cross-origin credential theft and CSRF variants without requiring any user interaction. A wildcard CORS configuration is therefore classified as a critical risk in this tool.

1.3 Project Objective

The objective of this project is to build a lightweight, browser-based, non-intrusive tool that:

- Performs passive header inspection — no active exploitation, no port scanning, no payload injection.
- Evaluates headers against current OWASP and Mozilla Web Security Guidelines standards.
- Produces a calibrated, human-readable security grade and actionable, copy-ready remediation directives scoped to only the headers that are missing.

- Exports a structured audit trail in JSON and CSV formats suitable for use in security review processes, compliance workflows, or CI/CD pipeline gates.

---

2. System Architecture & Methodology

2.1 High-Level Architecture

The application is a single-file Streamlit app (`WebSecurityScanner.py`) composed of four logical layers:

```
┌─────────────────────────────────────────────────────────┐
│                    Presentation Layer                   │
│  st.title / st.columns / st.markdown / st.expander      │
│  Custom GitHub-dark CSS via st.markdown(unsafe_allow_html)  │
├─────────────────────────────────────────────────────────┤
│                  Audit & Scoring Layer                  │
│  audit_url()  ──►  compute_grade()  ──►  grade_css_class()  │
├─────────────────────────────────────────────────────────┤
│                Remediation Content Layer                │
│  REMEDIATION_DIRECTIVES  ──►  st.expander + st.code()   │
├─────────────────────────────────────────────────────────┤
│                    Export Layer                         │
│  build_json_report()  /  build_csv_report()             │
│  st.download_button (JSON · CSV)                        │
└─────────────────────────────────────────────────────────┘
```

Layer 1 — Presentation: Renders the full UI using Streamlit widgets. A custom CSS block injected via `st.markdown` applies a GitHub-dark colour palette (`0d1117` background, `e6edf3` text) targeting Streamlit's internal `[data-testid="stAppViewContainer"]` attribute selector. Metric cards, header-row tiles, and status badges are all rendered as raw HTML strings passed through `st.markdown(..., unsafe_allow_html=True)`.

Layer 2 — Audit & Scoring: `audit_url()` issues a passive GET request, normalises response headers to lowercase, cross-references them against the `SECURITY_HEADERS` catalogue, evaluates the CORS directive, and feeds results into `compute_grade()`. This layer contains all security logic and returns a structured Python dict consumed by the other layers.

Layer 3 — Remediation Content: The `REMEDIATION_DIRECTIVES` constant maps each audited header to a pair of Nginx and Apache configuration strings. After a scan, the UI filters to headers that are absent, and renders an `st.expander` per missing header containing both server directives via `st.code()` with syntax highlighting.

Layer 4 — Export: `build_json_report()` and `build_csv_report()` serialise the audit result into downloadable strings, surfaced via `st.download_button` widgets with auto-stamped filenames.

2.2 Header Evaluation Engine

The audited header catalogue is defined in the module-level `SECURITY_HEADERS` constant — a Python dictionary mapping header names to descriptions and `critical` boolean flags:

Header	Description	Critical
`Strict-Transport-Security`	Enforces HTTPS connections (HSTS)	Yes
`Content-Security-Policy`	Restricts sources of executable scripts	Yes
`X-Content-Type-Options`	Prevents MIME-type sniffing	No
`X-Frame-Options`	Protects against clickjacking attacks	No
`Referrer-Policy`	Controls referrer information in requests	No
`Permissions-Policy`	Limits access to browser APIs / features	No

The `audit_url()` function executes the inspection:

1. `urlparse()` extracts the scheme from the raw URL prior to any redirect resolution.
2. `requests.get(url, timeout=10, allow_redirects=True)` issues a standard GET request, following redirect chains to the terminal response. A browser-like `User-Agent` string is injected to avoid `403` responses from bot-management platforms such as Cloudflare.
3. All response headers are normalised to lowercase in a single dictionary comprehension: `{k.lower(): v for k, v in resp.headers.items()}`, eliminating case-sensitivity issues without repeating `.lower()` at each lookup site.
4. Each entry in `SECURITY_HEADERS` is checked by key presence. If absent and `critical: True`, `critical_risks` is incremented.
5. The `Access-Control-Allow-Origin` header is retrieved and tested for a wildcard value (`"*"`). A confirmed wildcard increments `critical_risks` by 1 independently.
6. A structured result dict is returned, containing the per-header audit table, CORS evaluation, implementation counts, and the computed letter grade.

2.3 CORS Evaluation Mechanics

The CORS check inspects the `Access-Control-Allow-Origin` response header in the live GET response. Three outcomes are possible:

Outcome	Condition	Badge
NOT SET	Header is absent	⚠ WARN (yellow)
WILDCARD	Value is exactly `""` after whitespace stripping	✗ MISSING (red) — critical risk +1
RESTRICTED	Any other non-empty, non-wildcard value	☑ PRESENT (green)

The detection logic is:

```python
cors_value    = headers.get("access-control-allow-origin", "")
cors_wildcard = cors_value.strip() == ""
if cors_wildcard:
    critical_risks += 1
```

2.4 Grading Algorithm Mechanics

The letter grade is computed by `compute_grade(present, total, critical_risks)` using the following piecewise logic:

Step 1 — Critical risk floor check (evaluated first):

```
if critical_risks ≥ 2  →  Grade = F  (immediate; no further evaluation)
```

Any combination of two or more of the following triggers this floor: missing `Strict-Transport-Security`, missing `Content-Security-Policy`, or a wildcard CORS value. A site with even two critical gaps cannot achieve a passing grade, reflecting the principle that a single unmitigated XSS attack surface or an active MITM vector is disqualifying.

Step 2 — Ratio-based grading (only reached if critical_risks < 2):

```
ratio = present / total          (total = 6 audited headers)

ratio ≥ 0.90  →  A   (≥ 5.4 headers → effectively all 6 of 6)
ratio ≥ 0.70  →  B   (≥ 4.2 headers → effectively 5 of 6)
ratio ≥ 0.40  →  C   (≥ 2.4 headers → effectively 3 of 6)
ratio < 0.40  →  F
```

Grade colour mapping applied to metric cards:

Grade	CSS Class	Display Colour	Interpretation
A	`grade-a`	`3fb950` (green)	Strong security posture
B	`grade-b`	`58a6ff` (blue)	Good — minor gaps present
C	`grade-c`	`d29922` (amber)	Moderate — attention required
F	`grade-f`	`f85149` (red)	Poor or critical risk present

2.5 Export Module

`build_json_report(url, result)` serialises the scan result into a JSON object containing: `scanned_url`, `timestamp` (UTC ISO-8601), `grade`, `implemented`, `total`, `critical_risks`, `protocol`, `status_code`, and a `headers` array of per-header objects with `header`, `status`, `critical`, and `value` fields.

`build_csv_report(url, result)` produces a two-block CSV: a metadata preamble of key–value rows followed by a column-headed tabular section with one data row per audited header. The format opens cleanly in Microsoft Excel and Google Sheets.

Both are exposed via `st.download_button` widgets. Export filenames are auto-stamped using the sanitised target hostname and a UTC timestamp:

```
websec_github_com_20250711T143022Z.json
websec_github_com_20250711T143022Z.csv
```

---

3. Role of IBM Bob in Development

IBM Bob (https://www.ibm.com/products/bob) served as the AI developer copilot throughout the complete development lifecycle. Its contributions spanned UI architecture, HTTP logic, defect diagnosis, and content generation.

3.1 Scaffolding the Initial UI and Metric Cards

At project inception, Bob generated the foundational Streamlit layout structure. This included:

- `st.set_page_config(layout="wide")` to enable full-browser-width rendering.
- The four-column metric card row using `st.columns(4)`, with each column holding a custom HTML `<div class="metric-card">` component rendered via `st.markdown(..., unsafe_allow_html=True)`.
- The URL input and scan button in a `st.columns([5, 1])` ratio, giving the input field five times the width of the button.
- The idle/placeholder state branch (`if not run or not url:`) which pre-renders the four metric cards with `"—"` values before any scan runs, preventing a visually broken initial state.
- The GitHub-dark CSS palette injected via `st.markdown`, targeting Streamlit's internal `[data-testid="stAppViewContainer"]` attribute selector rather than `body` — necessary because Streamlit wraps content in a custom container that does not inherit `body` overrides in certain framework versions.

3.2 Custom User-Agent Injection to Avoid Bot-Filter False Positives

During development testing against real targets, certain domains behind Cloudflare, AWS WAF, and similar bot-management platforms returned `403 Forbidden` or `503 Service Unavailable` responses to any request carrying Python's default User-Agent string (`python-requests/2.x.x`). This caused the scanner to erroneously report network errors rather than header audit results against valid, live endpoints.

Bob identified this as a surface-level bot filter — one that inspects the `User-Agent` header and rejects non-browser clients — and proposed injecting a browser-like string in the `requests.get` call while keeping the scan entirely passive and non-intrusive:

```python
BROWSER_UA = (
"Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
"AppleWebKit/537.36 (KHTML, like Gecko) "
"Chrome/124.0.0.0 Safari/537.36"
)
```

```
resp = requests.get(
    url,
    timeout=10,
    allow_redirects=True,
    headers={"User-Agent": BROWSER_UA},
)
```

Bob also confirmed that `allow_redirects=True` is essential to follow HTTP → HTTPS redirect chains and evaluate the terminal response headers — the ones a real browser would receive and act upon.

3.3 Debugging the Streamlit CommonMark Badge Rendering Defect

After the badge HTML was authored, a persistent rendering defect manifested in the deployed Streamlit application: raw HTML tag text — `</div>`, `<span class="badge badge-missing">MISSING</span>` — leaked as visible plain text onto the page immediately above the CORS Configuration section, rather than rendering as styled badge components.

Bob diagnosed the root cause through systematic analysis of Streamlit's rendering pipeline:

> Streamlit's `st.markdown()` passes every string through a **CommonMark parser first**, before the HTML renderer processes it. When the HTML string is written as a multi-line triple-quoted f-string with indented content, the CommonMark parser encounters indented lines and whitespace-prefixed opening tags and interprets them as fenced code blocks or literal paragraph text — emitting them verbatim into the output. The HTML renderer subsequently receives only a fragment of the intended string, leaving the unparsed remainder as visible plain text on the page.

The prescribed fix was to collapse every `st.markdown` call containing a multi-element HTML block into a **single unbroken line string** — no embedded newlines, no leading indentation within the string. The HTML tag structure remained identical; only the whitespace formatting was eliminated.

Before (broken — multi-line triple-quoted string):

```python
st.markdown(
f"""<div class="header-row">
    <div>
<div class="header-name">{name}</div>
```

```
                    <div class="header-desc">{meta['desc']}</div>
                                </div>
                <span class="badge {badge_cls}">{badge_text}</span>
                            </div>""",
                        unsafe_allow_html=True,
                            )
                            ```
```

**After (correct — single-line string):**

```
                            ```python
                        st.markdown(
    f'<div class="header-row"><div><div class="header-name">{name}</div><div
  class="header-desc">{meta["desc"]}</div>{value_html}</div><span class="badge
                  {badge_cls}">{badge_text}</span></div>',
                        unsafe_allow_html=True,
                            )
                            ```
```

The `unsafe_allow_html=True` parameter was already correctly set throughout; the defect was exclusively the multi-line string format feeding the Markdown pre-processor before the HTML renderer ever ran.

### 3.4 Authoring the Nginx/Apache Remediation Snippets and Data Export Routines

Bob authored the entire `REMEDIATION_DIRECTIVES` constant — a Python dictionary mapping each of the seven audited headers to their production-ready Nginx and Apache configuration strings. Specific technical decisions guided by Bob included:

- **Using `always` in all Nginx `add_header` directives** — without this parameter, Nginx only emits these headers on 2xx success responses, leaving 4xx and 5xx error pages unprotected. A common misconfiguration omission in production deployments.
- **`max-age=31536000; includeSubDomains; preload`** for HSTS — the specific value required for submission to browser vendor preload lists (Chrome, Firefox, Edge), which harden the protection beyond what the per-site header alone provides.
- **`default-src 'self'; script-src 'self'; object-src 'none'`** as the baseline CSP — blocking all external resource loading and legacy plugin execution while remaining a deployable starting point for most applications.
- **`strict-origin-when-cross-origin`** for `Referrer-Policy` — the modern browser default since Chrome 85, but not universally deployed at the server level despite its importance.
```

- `camera=(), microphone=(), geolocation=()` for `Permissions-Policy` — explicitly disabling the three most commonly abused sensor APIs.

For the data export module, Bob drafted both `build_json_report()` and `build_csv_report()` in full, designed the JSON payload structure for SIEM and API consumption, and proposed the `st.columns([1, 1, 4])` column ratio for the export button row to prevent the two download buttons from stretching to full page width.

---

4. Testing & Experimental Results

4.1 Test Methodology

Each test case issues a passive GET request to the target URL with `allow_redirects=True` and a browser-like User-Agent string. The scan captures the terminal response headers after any redirect chain is resolved. No payloads are injected, no ports are scanned, no authentication tokens are submitted. The test is safe, legal, and non-intrusive against any publicly reachable endpoint.

4.2 Test Case: `https://github.com`

Scan Configuration:

Field	Value
Target URL	`https://github.com`
Request Method	GET (passive — no payload)
Redirect Following	Enabled (`allow_redirects=True`)
Request Timeout	10 seconds
User-Agent	Chrome 124 browser string

Summary Results:

Metric	Result
Overall Security Grade	**B**
Headers Implemented	**5 / 6**
Critical Risks Found	**0**
Scanned Protocol	**HTTPS**
HTTP Status Code	**200**

Header-by-Header Audit Table:

Header	Status	Observed Value / Notes
`Strict-Transport-Security`	☑ PRESENT	`max-age=31536000; includeSubdomains; preload`
`Content-Security-Policy`	☑ PRESENT	Declared directive sources returned
`X-Content-Type-Options`	☑ PRESENT	`nosniff`
`X-Frame-Options`	☑ PRESENT	`deny`
`Referrer-Policy`	☑ PRESENT	`origin-when-cross-origin, strict-origin-when-cross-origin`
`Permissions-Policy`	✖ MISSING	No header returned — triggers Hardening Recommendation
`Access-Control-Allow-Origin`	⚠ NOT SET	Header absent — not a wildcard, not a critical risk

Grade Derivation (step-by-step):

```
present        = 5   (STS, CSP, X-Content-Type-Options, X-Frame-Options,
                      Referrer-Policy)
total          = 6
ratio          = 5 / 6 = 0.8333  →  83.3 %
critical_risks = 0

Step 1: critical_risks (0) < 2   →  F floor not triggered
Step 2: ratio (0.8333) ≥ 0.70    →  Grade = B
```

`Permissions-Policy` is flagged `critical: False` in the `SECURITY_HEADERS` catalogue, so its absence does not increment `critical_risks`. The `Access-Control-Allow-Origin` header is absent but not wildcarded, so the CORS critical-risk counter also stays at 0. With `critical_risks = 0` and a ratio of 83 %, the algorithm assigns Grade **B**.

4.3 Hardening Recommendation Generated

Because `Permissions-Policy` was the sole missing header, the scanner rendered a single collapsible `st.expander` under the 🔧 **Hardening Recommendations** section containing the following ready-to-copy directives:

Nginx — add inside the `server {}` or `location {}` block:

```nginx
add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;
```

Apache — add inside `<VirtualHost>` or `.htaccess` (requires `mod_headers`):

```apache
Header always set Permissions-Policy "camera=(), microphone=(), geolocation=()"
```

4.4 Export Output

The scanner produced two downloadable files via the **Export Report** section:

```
websec_github_com_<UTC-timestamp>.json
websec_github_com_<UTC-timestamp>.csv
```

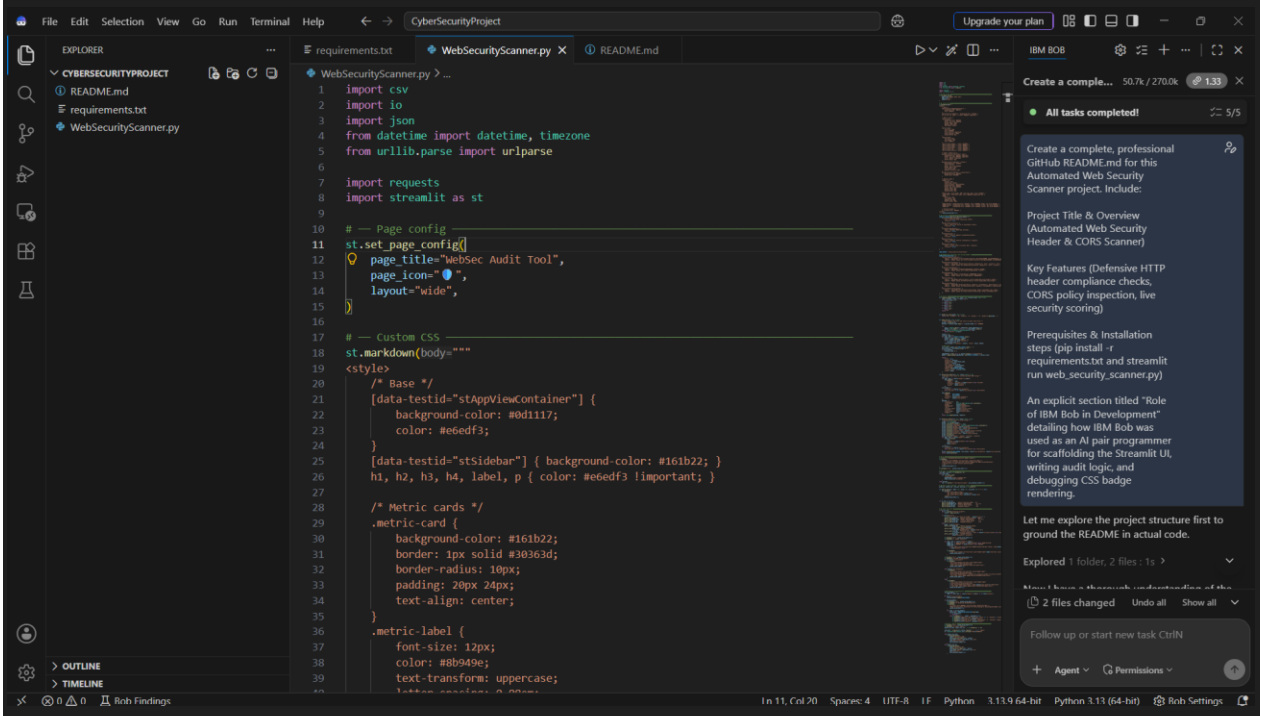
The JSON report payload structure:

```json
{
  "scanned_url": "https://github.com",
  "timestamp": "2026-10-24T10:30:00+00:00",
  "grade": "B",
  "implemented": 5,
  "total": 6,
  "critical_risks": 0,
  "protocol": "HTTPS",
  "status_code": 200,
  "headers": [
    { "header": "Strict-Transport-Security", "status": "present", "critical": true, "value": "max-age=31536000; includeSubdomains; preload" },
    { "header": "Content-Security-Policy", "status": "present", "critical": true, "value": "..." },
    { "header": "X-Content-Type-Options", "status": "present", "critical": false, "value": "nosniff" },
```

```
    { "header": "X-Frame-Options",           "status": "present", "critical":
                        false, "value": "deny" },
    { "header": "Referrer-Policy",           "status": "present", "critical":
            false, "value": "strict-origin-when-cross-origin" },
    { "header": "Permissions-Policy",        "status": "missing", "critical":
                        false, "value": "" },
    { "header": "Access-Control-Allow-Origin","status": "not_set","critical":
                        false,  "value": "" }
                            ]
                           }
                          ```
```

### 4.5 Screenshots

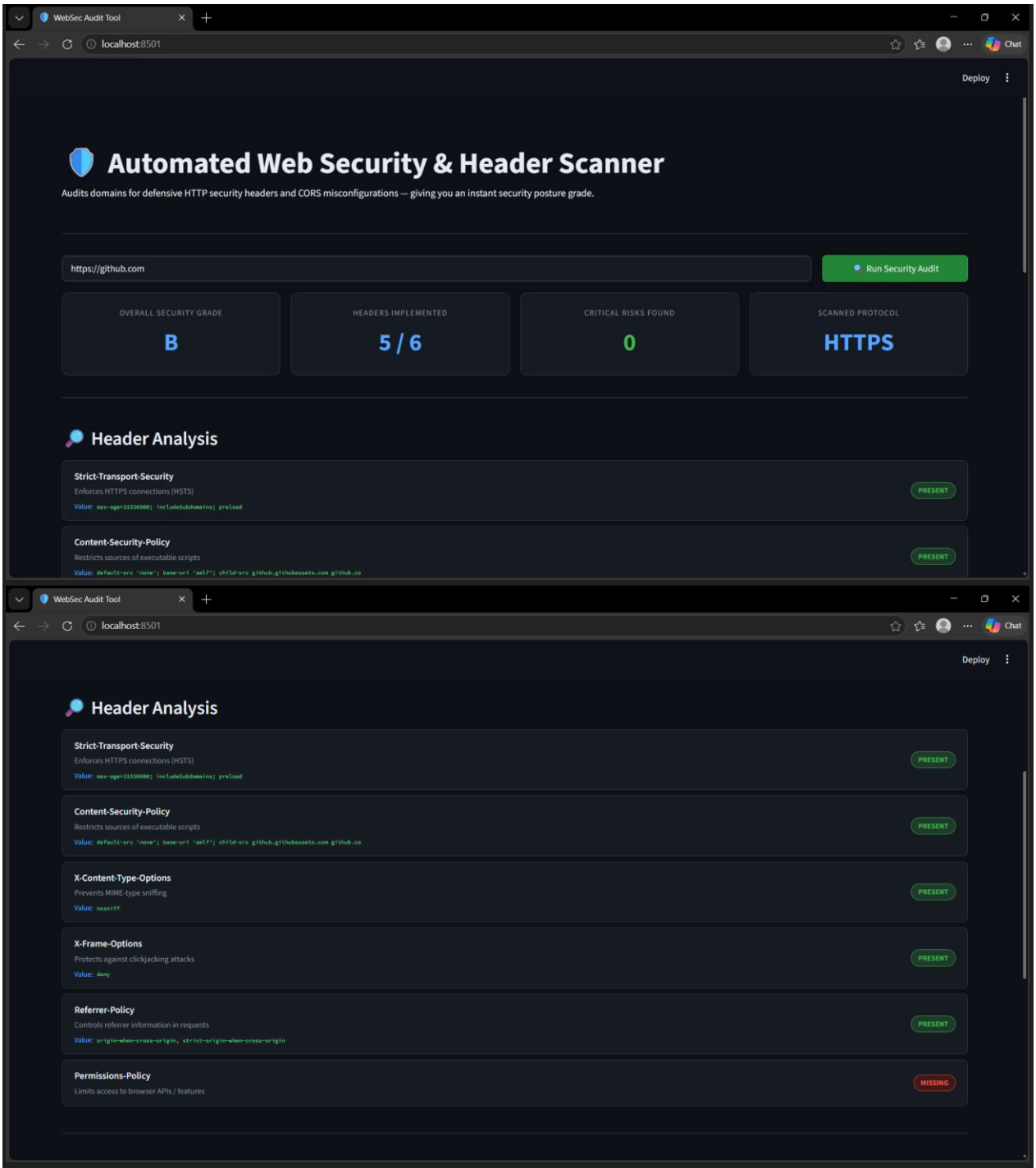
---




**[Figure 1: VS Code Workspace and IBM Bob Pair Programming Session]**

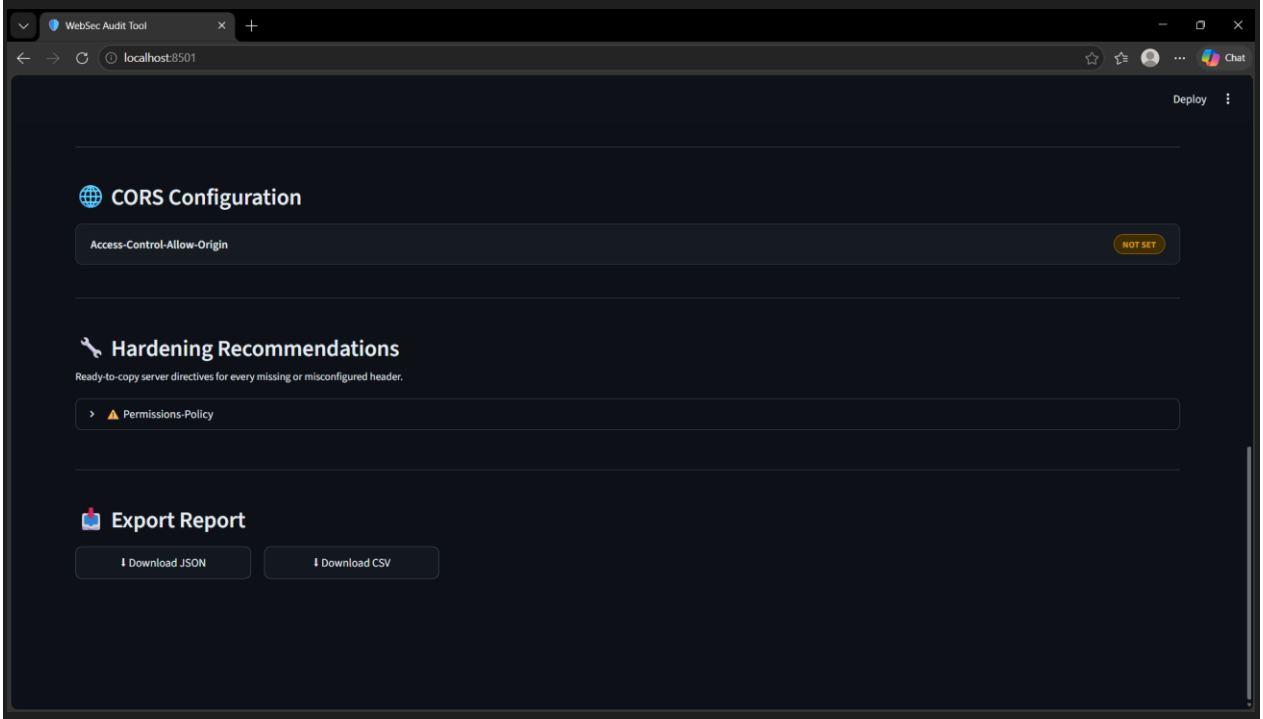
*IBM Bob AI pair programming session in the chat panel alongside the VS Code editor displaying `WebSecurityScanner.py`. The conversation illustrates the scaffolding of the Streamlit dashboard layout and the diagnosis of the CommonMark badge rendering defect.*

---
```



[Figure 2,3: Streamlit Dashboard Top Section — Results on github.com]

The live scan results for `https://github.com` displayed in the Streamlit app. Four metric cards show Grade B, 5/6 headers implemented, 0 critical risks, and HTTPS protocol. The Header Analysis section below shows five green PRESENT badges and one red MISSING badge for `Permissions-Policy`.



[Figure 4: Hardening Recommendations and JSON/CSV Export Controls]

The Hardening Recommendations expander for `Permissions-Policy` is open, showing the ready-to-copy Nginx `add_header` and Apache `Header always set` directives. Directly below, the Export Report section displays the Download JSON and Download CSV buttons in the left-aligned column layout.

---

5. Remediation Directives & Hardening Guide

The following configuration blocks represent the complete set of server directives needed to remediate all seven audited security controls. They are sourced directly from the `REMEDIATION_DIRECTIVES` constant in `WebSecurityScanner.py` and represent production-grade, standards-compliant values.

5.1 Nginx — Full Security Header Block

Add the following inside the `server {}` block of your Nginx virtual host configuration (typically `/etc/nginx/sites-available/your-site.conf`), then reload Nginx with `sudo nginx -s reload`:

```nginx
── HTTP Security Headers ─────────────────────────────────────────

Enforce HTTPS for 1 year; include subdomains; qualify for browser preload list
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;

Restrict all resource sources to same origin; block legacy plugin execution
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none';" always;

Prevent MIME-type sniffing attacks
add_header X-Content-Type-Options "nosniff" always;

Block cross-origin iframe embedding (anti-clickjacking)
add_header X-Frame-Options "SAMEORIGIN" always;

Send only the origin component of the URL in cross-origin Referer headers
add_header Referrer-Policy "strict-origin-when-cross-origin" always;

Disable camera, microphone, and geolocation APIs for this origin
add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;

Restrict CORS to a specific trusted origin (replace with your actual origin)
add_header Access-Control-Allow-Origin "https://yourdomain.com" always;
```

> **Critical note on `always`:** The `always` parameter causes Nginx to include these headers on all response codes, including `4xx` and `5xx` error pages. Without it, Nginx only sends these headers on `2xx` success responses, leaving error pages entirely unprotected — a common production misconfiguration.

5.2 Apache — Full Security Header Block

Add the following inside the `<VirtualHost>` directive of your Apache configuration. Requires `mod_headers` to be enabled (`sudo a2enmod headers && sudo systemctl reload apache2`):

```apache
── HTTP Security Headers ──────────────────────────────────────────

Enforce HTTPS for 1 year; include subdomains; qualify for browser preload list
Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"

Restrict all resource sources to same origin; block legacy plugin execution
Header always set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src 'none';"

Prevent MIME-type sniffing attacks
Header always set X-Content-Type-Options "nosniff"

Block cross-origin iframe embedding (anti-clickjacking)
Header always set X-Frame-Options "SAMEORIGIN"

Send only the origin component of the URL in cross-origin Referer headers
Header always set Referrer-Policy "strict-origin-when-cross-origin"

Disable camera, microphone, and geolocation APIs for this origin
Header always set Permissions-Policy "camera=(), microphone=(), geolocation=()"

Restrict CORS to a specific trusted origin (replace with your actual origin)
Header always set Access-Control-Allow-Origin "https://yourdomain.com"
```

5.3 CSP Refinement Notes

The baseline CSP (`default-src 'self'; script-src 'self'; object-src 'none'`) is intentionally conservative. The following refinements are commonly required in production:

- **External fonts (Google Fonts):** add `style-src 'self' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com;`
- **Analytics scripts:** use a cryptographic nonce (`script-src 'nonce-{random}'`) rather than a domain allowlist, to prevent injection via the analytics provider's infrastructure.
- **Legacy inline JavaScript:** migrate to nonce-based CSP rather than permitting `'unsafe-inline'` globally, which defeats the entire purpose of a CSP.

---

6. Conclusion & Future Scope

6.1 Conclusion

This project successfully delivers a non-intrusive, browser-based security auditing tool that covers the most impactful defensive HTTP header controls documented by OWASP and Mozilla's Web Security Guidelines. The tool performs a passive GET-based inspection, computes a weighted letter-grade security posture score, surfaces per-header hardening directives scoped exclusively to missing controls, and exports a structured audit trail in JSON and CSV formats — all within a dependency-light, ~440-line Python application requiring no server infrastructure beyond a local Streamlit process.

Development in partnership with IBM Bob demonstrated measurable productivity gains: the AI copilot reduced the time required to scaffold a functional multi-column dashboard layout, correctly identify and resolve a non-obvious rendering defect caused by Streamlit's CommonMark pre-processing pipeline, and generate a complete, standards-compliant Nginx/Apache directive set covering seven security controls.

The test case against `https://github.com` — a security-conscious, production-grade target operated by one of the world's largest software platforms — returned Grade **B** with 5 of 6 headers present and 0 critical risks. This validates that the tool produces calibrated, realistic assessments: even industry-leading sites have minor hardening gaps, and the grading scale correctly distinguishes between negligent postures and responsible-but-imperfect ones.

6.2 Future Scope

Enhancement	Description	Priority
TLS / SSL Certificate Inspection	Check certificate validity, expiry date, cipher suite strength (deprecate RC4, 3DES, TLS 1.0/1.1), and OCSP stapling status using Python's `ssl` module or `pyOpenSSL`	High
CI/CD Pipeline Integration	Package the audit engine as a CLI tool (`python websec_audit.py --url <target> --fail-on-grade C`) for use as a security gate in GitHub Actions, GitLab CI, or Jenkins	High
Cookie Security Attribute Audit	Inspect `Set-Cookie` response headers for missing `HttpOnly`, `Secure`, and `SameSite` attributes — common causes of session hijacking and CSRF	Medium

CSP Policy Strength Analysis	Parse the CSP header value and flag dangerous directives (`unsafe-inline`, `unsafe-eval`, wildcard source allowlists) rather than treating any non-empty CSP value as a full pass	Medium
Historical Trend Tracking	Persist scan results to a local SQLite database and render a time-series line chart (`st.line_chart`) showing security grade evolution across repeated scans of the same target	Medium
Multi-URL Batch Mode	Accept a list of URLs via a CSV upload or newline-separated text area, scan all targets concurrently using `concurrent.futures.ThreadPoolExecutor`, and produce a consolidated ranked report	Low
Subdomain Enumeration	Accept a root domain, enumerate common subdomains via a wordlist, audit each independently, and surface a per-subdomain summary table in the UI	Low
Authenticated Endpoint Support	Allow users to supply a session cookie or Bearer token header so the tool can audit authenticated endpoints that return different security headers behind a login wall	Low

---

7. References

1. OWASP Foundation. *OWASP Secure Headers Project.* https://owasp.org/www-project-secure-headers/
2. Mozilla. *Mozilla Web Security Guidelines — HTTP Headers.* https://infosec.mozilla.org/guidelines/web_security
3. IETF RFC 6797. *HTTP Strict Transport Security (HSTS).* https://datatracker.ietf.org/doc/html/rfc6797
4. IETF RFC 7034. *HTTP Header Field X-Frame-Options.* https://datatracker.ietf.org/doc/html/rfc7034
5. W3C / WHATWG. *Fetch Standard — CORS Protocol.* https://fetch.spec.whatwg.org/cors-protocol
6. W3C. *Content Security Policy Level 3.* https://www.w3.org/TR/CSP3/
7. Google Chromium. *Content Security Policy (CSP) Reference.* https://developer.chrome.com/docs/privacy-security/csp
8. W3C. *Permissions Policy Specification.* https://www.w3.org/TR/permissions-policy/
9. Scott Helme. *SecurityHeaders.com — HTTP Security Header Analysis Tool.* https://securityheaders.com
10. Streamlit Inc. *Streamlit Documentation — st.markdown.* https://docs.streamlit.io/library/api-reference/text/st.markdown

11. Python Software Foundation. *Requests: HTTP for Humans — Documentation.* https://requests.readthedocs.io/
12. IBM. *IBM Bob — AI Developer Copilot.* https://www.ibm.com/products/bob

---

End of Report